

A DBMS Project on Library Management System

Submitted by

Uttkarsh, Alok, Sahil, Abhishek, Disha Bhagyashri

V Sem B.Tech

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

Department of Computer Science and Engineering

PW Institute of Innovation

June 22, 2026

Abstract

The Library Management System is a computerized system used to store and retrieve information and conduct transactions related to library operations. It is aimed at exposing the importance of digital management systems to maintain books and enhance a user's convenience to borrow and return books seamlessly.

The website made with regard to this project has two main perspectives: the administrative/librarian perspective and the library member's perspective. The front end is a dynamic dashboard built for ease of use, while the back end handles data integrity and logic. It allows librarians to access the database to add new books, register new library members, and issue books. The system displays all available books, their prices, authors, and available copies.

To borrow a book, the system links the member's profile with the book's details, recording the issue date and due date automatically. The main problem this project aims at eradicating is to eliminate manual errors that can be made while managing thousands of books and physical member records, enabling a modern, error-free, and highly aesthetic online dashboard.

CONTENTS

1. **INTRODUCTION**
 - 1.1 Access Specification
2. **REQUIREMENT ANALYSIS**
 - 2.1 Functional Requirements
 - 2.2 Non-Functional Requirements
 - 2.2.1 Performance/Time
 - 2.2.2 Usability
 - 2.2.3 Security
 - 2.2.4 Reliability and Availability
3. **CONCEPTUAL MODELS**
 - 3.1 Entity-Relationship Diagram
 - 3.2 Relational Schema
4. **SYSTEM DESIGN**
 - 4.1 Design Goals
 - 4.2 Design Development
 - 4.3 Detailed Design Understanding
 - 4.3.1 Data Gathering
 - 4.3.2 Merging Front-end and Back-end
5. **CONCLUSION**
6. **REFERENCES**

1. INTRODUCTION

In an era where digital transformation has taken over traditional paper-based methods, institutions are increasingly adopting digital management systems. Libraries, holding thousands of books and serving numerous members, require a robust mechanism to track their assets efficiently.

This document aims at designing a Library Management System that acts as a portal for tracking book inventories, registering members, and processing borrow/return transactions. Therefore, it is important to make a good database design that accurately reflects the real-world interactions within a library.

The system provides a portal to view all books and members. In order to keep track of a borrow transaction, a member is registered with the following details: Name, Address, Membership Type, and Join Date. The system automatically calculates their membership expiry. Similarly, books are recorded with their Publisher, Price, and Available Copies.

1.1 Access Specification

The primary user of this specific system architecture is the Librarian/Admin. The Admin is responsible for:

- Adding new books and tracking available copies.
- Registering new members and managing their membership status.
- Issuing books to members and setting due dates.
- Viewing real-time statistics of the most popular books and active borrow transactions.

2. REQUIREMENT ANALYSIS

2.1 Functional Requirements

The web application provides a comprehensive dashboard for library operations.

- **Book Management:** The user can view a real-time list of all available books in the library. They can add new books by entering the title, author, price, and linking it to a publisher.
- **Member Management:** The user can view all registered members, their membership types, and expiry dates. New members can be added seamlessly via a form.
- **Borrow Transactions:** The system allows issuing a book to a member. The user selects the book and member from a dropdown, sets an issue and due date, and upon confirmation, the available copies of the book are automatically decremented.
- **Popularity Tracking:** A dedicated section dynamically ranks the most popular books based on how many times they have been borrowed.

2.2 Non-Functional Requirements

2.2.1 Performance/Time

The query processing and retrieval of the results (such as fetching 100+ members and their borrow histories) take merely milliseconds thanks to the optimized MySQL relational joins.

2.2.2 Usability

Usability is a crucial feature. The web interface utilizes a modern, sleek white theme with clean typography and dynamic JavaScript rendering, ensuring anyone using it can navigate without prior training.

2.2.3 Security

Database security is maintained using connection pooling and parameterized SQL queries to prevent SQL Injection attacks when adding new books or members.

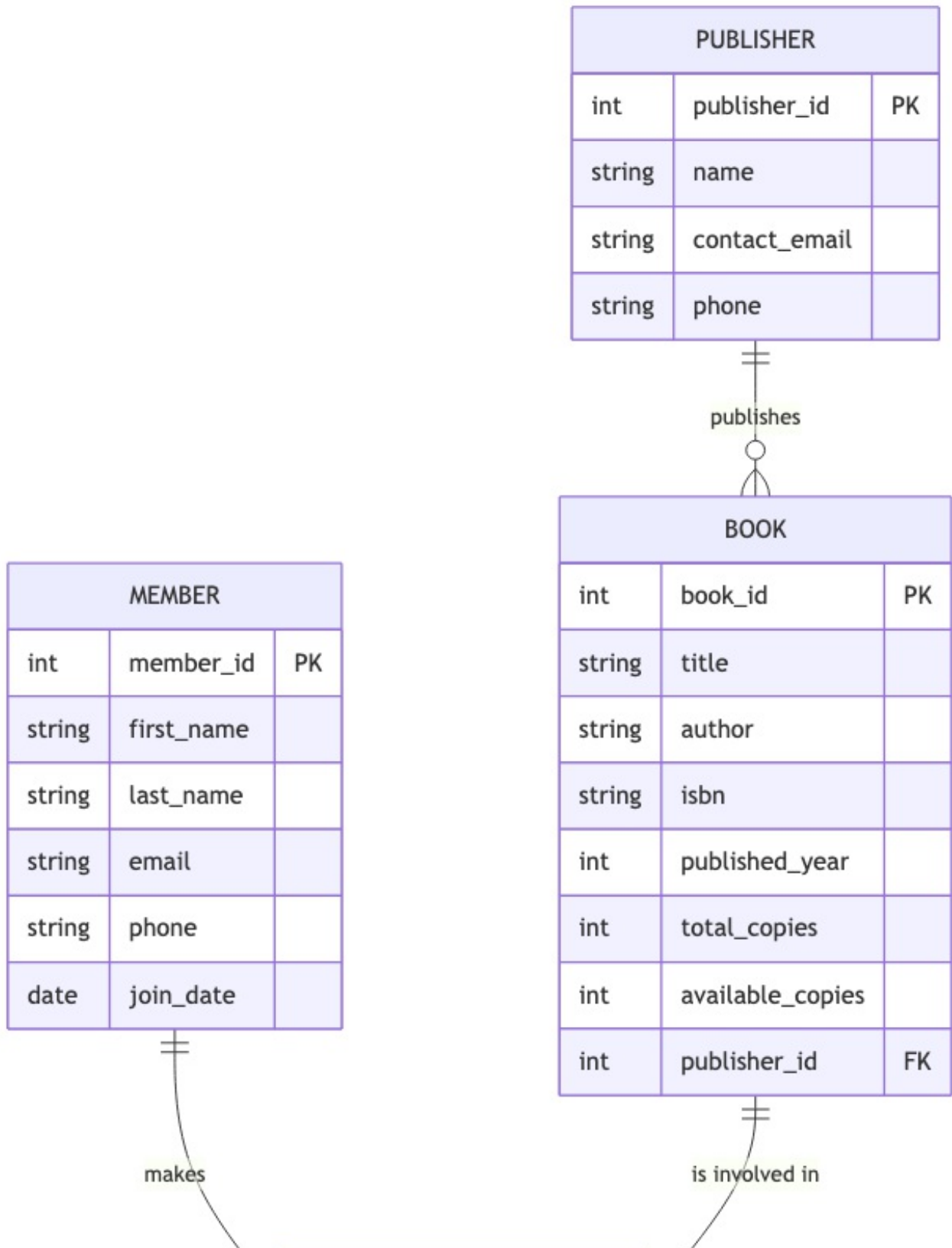
2.2.4 Reliability and Availability

This system ensures that the calculations for available book copies remain perfectly accurate even during concurrent issue transactions.

3. CONCEPTUAL MODELS

3.1 Entity-Relationship Diagram

The following ER Diagram maps the conceptual relationships between Books, Publishers, Members, and Transactions.



BORROW_TRANSACTION		
int	transaction_id	PK
int	book_id	FK
int	member_id	FK
date	borrow_date	
date	due_date	
date	return_date	
enum	status	

3.2 Relational Schema

The logical definition of tables derived from the ER Diagram.

Publishers:

publisher_id (PK) | name | contact_email | phone

Books:

book_id (PK) | title | author | isbn | published_year | total_copies | available_copies | publisher_id (FK)

Members:

member_id (PK) | first_name | last_name | email | phone | join_date

Borrow_Transactions:

transaction_id (PK) | book_id (FK) | member_id (FK) | borrow_date | due_date | return_date | status

4. SYSTEM DESIGN

The design of the system is divided into Front-end design and Back-end integration.

4.1 Design Goals

- The design must be rendered using a RESTful Node.js Express backend and a MySQL database.
- The front-end must use HTML5, modern CSS3 with a sleek white theme, and vanilla JavaScript.
- The code ought to be readable and well-designed, meeting all functional requirements.

4.2 Design Development

The front end runs on any standard web browser. It uses Fetch APIs to dynamically communicate with the backend. The backend runs on Node.js and relies on the mysql2 driver to execute relational queries.

4.3 Detailed Design Understanding

4.3.1 Data Gathering

To simulate a real-world scenario, the database was populated with large sample datasets.

- 15 Publishers (e.g., Penguin Books, HarperCollins)
- 105 Books (e.g., Clean Code, The Great Gatsby)
- 101 unique Members.
- 28 active Borrow Transactions.

4.3.2 Merging Front-end and Back-end

The application acts as a Single Page Application (SPA). The main view displays dynamic tables that fetch data directly from the backend.

Dashboard & User Interface Features:

1. **Dynamic Tables:** "Available Books" and "Library Members" tables are populated immediately on page load via GET /api/books and GET /api/members.
2. **Issue Book Module:** Features dropdowns dynamically linked to the database IDs to ensure referential integrity when inserting into Borrow_Transactions.
3. **Real-time Availability:** When a book is issued, the UI and database automatically deduct the available copies.

5. CONCLUSION

This project successfully achieved its goal of building an online Library Management System. The Admin can seamlessly manage thousands of books, members, and transactions. By utilizing an optimized MySQL schema with strict Foreign Key constraints and a robust Node.js backend, the system guarantees data integrity. The sleek white theme and responsive dashboard provide excellent user satisfaction and ease of use.

6. REFERENCES

- [1] Node.js and Express Documentation - <https://nodejs.org/>
- [2] MySQL Relational Database Concepts - "Fundamentals of Database Systems - By Elmasri & Navathe"
- [3] HTML and CSS Basics - <https://www.w3schools.com/>